Computer Vision

Nicola Conci
a.y. 2025/2026
University of Trento

Davide Donà

February 2026

Notes Repository

Contents

1 Definition of Computer Vision **1**

2 Images and Videos **1**

2.1 Processing Chain 1

2.2 Digital Images 2

2.2.1 Sampling 2

2.3 Colors 3

2.3.1 RGB 3

2.3.2 Problems of RGB 3

2.3.3 YCbCr 4

2.3.4 HSV 5

2.3.5 Bayer Pattern 5

2.3.6 Quantization 5

2.4 Videos 6

2.4.1 Important Features 6

2.5 Histogram 6

2.5.1 Information from the histogram 7

2.6 Simple Operations on the Histogram 7

2.6.1 Stretching 7

2.6.2 Equalization 8

2.7 Edge Extraction 8

2.7.1 Gradient based approach 8

2.7.2 Practical Example - Sobel Operators 9

2.7.3 Convolution 9

2.8 Smoothing / Enhancement 10

2.8.1 Low-pass filter 10

2.8.2 Gaussian filter 11

2.8.3 Median filtering 11

3 Morphology Filters **11**

1 Definition of Computer Vision

Computer Vision is the science and technology of machines that **see**, where "see" means being able to extract information from an image, necessary to perform a task.

2 Images and Videos

Before introducing the concept of images and videos, we first need to define the concept of **signal**, how to represent it and how to process it.

2.1 Processing Chain

The **processing chain** of an image can be represented by the following steps:

- **Acquisition**: The process of transforming a real-world physical signal into a digital one, by means of a **sensor**. In this process, we essentially transfer a portion of the real 3D world into a 2D image. This process is affected by **noise**, which can be added by both the real world and the sensor itself.

  Once the signal is acquired, it is transformed using a standard format to a discrete signal, by applying both a **sampling** and a **quantization** process.

- **Enhancement**: The process of applying a transformation to the signal, by applying a **filter** to it, in order to make the picture more suitable for our goal.

- **High-level processing**: The process of extracting information from the signal. This is what we will see in the next chapters.

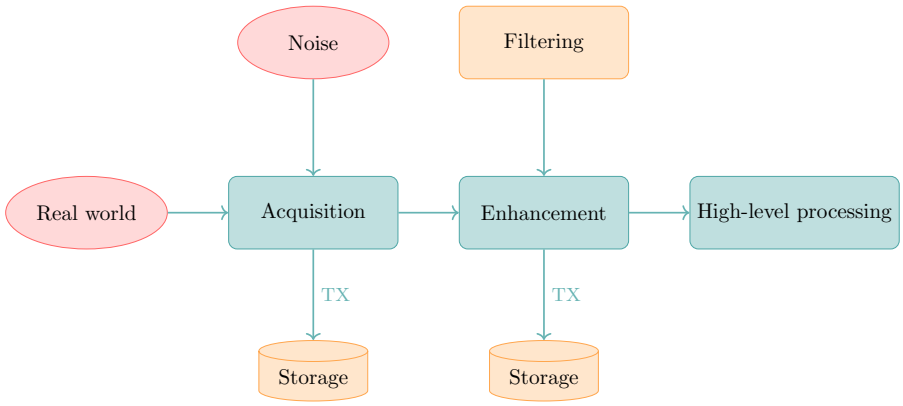

Figure 1: Processing chain of an image.

2.2 Digital Images

A digital image can be seen as a collection of 2D coordinates, called **pixels** (picture elements). Each pixel represents a portion of the real world and can be **gray-scale** (black and white, 1 channel) or **color** (typically 3 channels).

2.2.1 Sampling

As we said before, to transform a real-world signal into a digital one, we need to apply a sampling process. When considering video signals, they need to be sampled in all three dimensions: the two spatial dimensions (x and y) and the temporal dimension (t).

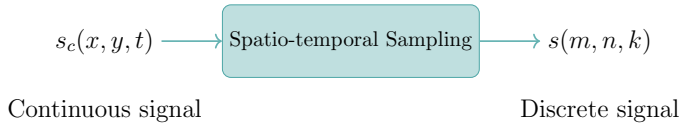

Figure 2: Spatio-temporal sampling of a video signal.

This process is called **spatio-temporal sampling**, and it transforms a continuous signal $s_c(x, y, t)$ into a discrete one $s(m, n, k)$, where m, n and k are the indices of the spatial and temporal dimensions, respectively.

The output of this process is a 2D matrix of pixels. Once we have this, we can manipulate it using many techniques. Pixels are numbered starting from the top-left corner, and the value of a pixel in certain position (c, r) for a given image I is denoted as $I(c, r)$, where c is the column index and r is the row index. The value of a pixel is typically normalized in the range $[0, 1]$.

2.3 Colors

Color is an attribute that humans associate with objects in the real world. Mathematically, we can define it as a combination of different **wavelengths**. Different representations are used to represent colors; we will discuss the most common ones in the following chapters.

2.3.1 RGB

One of the most common representations of colors is the **RGB** (Red, Green, Blue) one. In the *addictive color model*, starting from a black image, we can add different amounts of red, green and blue to it, creating a wide range of colors.

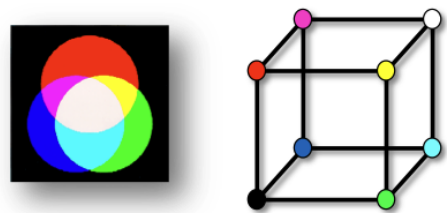

Figure 3: Addictive color model.

We can also define the opposite process, called *subtractive color model*. In this model, we start from a white image, and we subtract different amounts of cyan, magenta and yellow to it (the **primary colors**), creating a wide range of colors.

2.3.2 Problems of RGB

The RGB representation is not the optimal one. In fact, if we analyze how the human eye perceives colors, we can see that it is a lot more sensitive to some colors than others. For example, we are more sensitive to luminance than to color information, and less sensitive to color variations. This implies that a lot of information is redundant in the RGB representation, and other representations may be more effective.

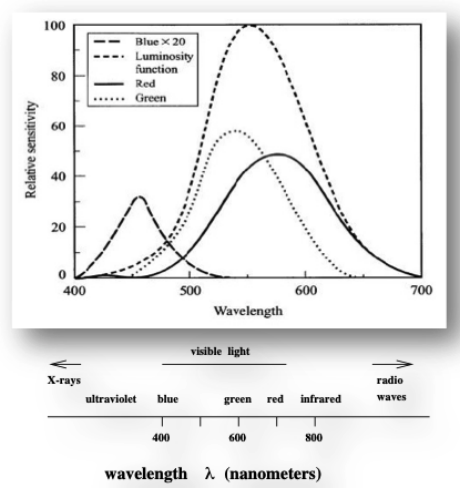

Figure 4: Sensitivity of the human eye to different colors.

Also, if we take an RGB image and separate its three components, we can see that they are highly correlated, meaning that they contain a lot of redundant information. This is also not ideal for image processing tasks.

2.3.3 YCbCr

The **YCbCr** representation is a color space that separates the luminance (Y) from the color information (Cb and Cr). This representation is more suitable for image compression (used in JPEG and MPEG standards) and transmission, as it allows to reduce the amount information, without losing too much quality to the human eye.

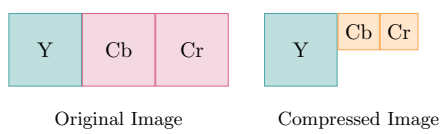

Figure 5: YCbCr compression example.

As we can see from the figure, by down-sampling the Cb and Cr components by a factor of 2, we can reduce the size of the image in half, without any noticeable difference in quality to the human eye.

2.3.4 HSV

The **HSV** (Hue, Saturation, Value) representation is another color space that separates the hue (H) from the saturation (S) and value (V). This representation is more suitable for computer graphics and image processing.

2.3.5 Bayer Pattern

In the acquisition process, the sensor of a camera must be able to capture the light information for all the tree components in order to create a color image. The perfect solution would be to have three different sensors, one for each color component, but this is not practical and too expensive.

The most common solution is to use a **Bayer pattern**, which is a mosaic of color filters that are placed on top of the sensor. They are arranged in a specific pattern, to correctly exploit the sensitivity of the human eye to different colors. The pattern is composed as follows:

- 50% of the pixels are green, to capture the luminance information, which is the most important for the human eye.
- 25% of the pixels are red, to capture the red color information.
- 25% of the pixels are blue, to capture the blue color information.

This pattern allows to capture the color information in a single shot, without the need for multiple sensors, and it is widely used in digital cameras.

2.3.6 Quantization

Once we have the discrete signal, we need to quantize it, meaning that we need to assign a finite number of values to the pixel values. The most common quantization is to use 8 bits per pixel (bpp), which allows to represent 256 different values for each channel.

This is a good compromise between the quality of the image and the amount of data that we need to store and transmit, while also being handy for processing, as each pixel value can be represented by only one byte of memory. Using less than 8 bpp would result in a loss of quality, introducing the problem of **contouring**.

After quantization, we can represent the pixel values as integers in the range $[0, 255]$, where 0 represents black and 255 represents the maximum intensity of the color. For colored images, we can represent the pixel values as a tuple of three integers, one for each channel (R, G, B), in the range $[0, 255]$.

2.4 Videos

While static images provide information about static scenes, they lack motion information such as temporal evolution and dynamic interactions. Videos, on the other hand, capture a sequence of images over time, providing a richer representation of the world.

A video can be seen as a sequence of 2D images, that represent the projection of a **moving** 3D scene onto the video camera sensor. Usually videos have a lower resolution than images, as they would be too heavy to store and transmit otherwise. Each second of video is composed by a certain number of frames (typically 24, 30 or 60), and each frame is a 2D image. For videos lasting several minutes, the amount of data can be huge. Also, the information carried by videos is mostly given by the motion, and not by the color information.

2.4.1 Important Features

Once the images are acquired, we can extract some important features from them. Usually we are interested in:

- **Color** information and its distribution in the image. Described by the histogram of the image;
- **Shape** information, given by the contours and edges in the image. Described by the edges of the image.

If we take in consideration videos, we are interested how the two previous features evolve over time. This has many challenges

2.5 Histogram

A histogram is a simple way to describe the color distribution of an image. It can be seen as a probability distribution of the pixel values in the image. For a gray-scale image, the histogram is a 1D array of 256 values, where each value represents the number of pixels in the image that have the corresponding intensity value.

For an $M \times N$ grayscale image I, we have the following:

- The sum over all the histogram values is equal to the total number of pixels in the image:
$$\sum_{i=0}^{255} H(i) = M \times N$$

- We can represent the histogram as a probability distribution by normalizing it:

$$hist(i) = \frac{\#pixels : I(c,r) = i}{M \times N}$$

This can obviously be extended to color images, by computing the histogram for each channel separately.

2.5.1 Information from the histogram

The histogram can give us some information about the image, such as:

- The **mean** intensity of the image, giving us an idea of how bright or dark the image is;
- The **distribution** of colors in the image.

This can be seen as a signature of the image, and it can be used for very different tasks.

2.6 Simple Operations on the Histogram

Given the histogram of an image, we can apply some simple operations to it, in order to enhance the image or to extract some information from it.

2.6.1 Stretching

One of the most common operations that we can do on an image is **stretching**. This is a simple operation that allows to enhance the contrast of the image, by stretching the histogram to cover the entire range of pixel values.

To do this, we usually apply a piece-wise linear transformation to the pixel values. We define two thresholds, t_1 and t_2, and we apply the following transformation to the pixel values:

- If $I(c,r) < t_1$, we set $I'(c,r) = 0$. This means that all the pixel values below t_1 are set to black.
- If $I(c,r) > t_2$, we set $I'(c,r) = 255$. This means that all the pixel values above t_2 are set to white.
- If $t_1 \leq I(c,r) \leq t_2$, we create a linear transformation that maps the range $[t_1, t_2]$ to the range $[0, 255]$:

The first and the second operations are called **clipping**, while the third one is called **scaling**.

2.6.2 Equalization

Ideally, we would like to have a histogram that is as flat as possible, meaning that all the pixel values are equally represented in the image. This can be achieved by applying a transformation to the image called **equalization**. This transformation is defined as follows:

- **Cumulative histogram**: We first compute the cumulative histogram of the image, which is defined as:

$$CHist_I(i) = \sum_{k=0}^{i} hist(k)$$

- **Equalization transformation**: We then apply the following transformation to the pixel values:

$$hist_{equalized}(i) = \frac{CHist_I(i) - CHist_I(0)}{M \times N} \times 255$$

2.7 Edge Extraction

Edge extraction is a fundamental operation in image processing, as it allows to extract the shape information from the image. An edge can be seen as a **sharp change** in the brightness of the image.

The process of edge extraction can be seen as a two-step process:

- **Edge detection**: determine the direction and the strength of edges for each pixel in the image. This can be done using different techniques, such as gradient-based methods or Laplacian-based methods.
- **Binarization**: Once we have the edge strength for each pixel, we need to apply a threshold to it, in order to determine which pixels are edges and which are not (binary image of edges).

2.7.1 Gradient based approach

In the gradient-based approach, the goal is to find maxima and minima pixels in the image, along a certain direction. This can be formalized as:

$$\frac{\partial f}{\partial r} = \frac{\partial f}{\partial x}\frac{\partial x}{\partial r} + \frac{\partial f}{\partial y}\frac{\partial y}{\partial r} = f_x cos(\theta) + f_y sin(\theta) \tag{2.1}$$

Where f is the image, r is the direction of the edge, and θ is the angle of the edge with respect to the x-axis. We can then derivate it again with respect to θ to find the maxima and minima directions:

$$\frac{\partial}{\partial \theta}\left(\frac{\partial f}{\partial r}\right) = -f_x sin(\theta_g) + f_y cos(\theta_g) = 0 \implies \theta_g = arctan\left(\frac{f_y}{f_x}\right) \tag{2.2}$$

Where θ_g is the direction of the edge. We can then compute the edge strength as the magnitude of the gradient:

$$\sqrt{f_x^2 + f_y^2} \tag{2.3}$$

2.7.2 Practical Example - Sobel Operators

In practice, **FIR filters** are typically adopted to compute the gradient of the image, as they are computationally efficient and can be easily implemented using convolution.

The most common FIR filters for edge detection are the **Sobel operators**, which are defined as follows:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \tag{2.4}$$

For each pixel in the image, we apply the two filters to it and compute the edge strength as the magnitude of the gradient:

$$D = \sqrt{D_x^2 + D_y^2} \tag{2.5}$$

Where D_x and D_y are the results of the convolution of the image with the G_x and G_y filters, respectively.

2.7.3 Convolution

In order to apply the filters to the image, we need to perform a convolution operation between an image f and a filter h. In 2D, the convolution is defined as follows:

$$g(x, y) = (f * h)(x, y) = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} f(i, j)h(x - i, y - j) \tag{2.6}$$

In simpler words, we first flip the filter h 180 degrees counter-clockwise, and then we slide it over the image f, computing the sum of the products of the overlapping pixels at each position.

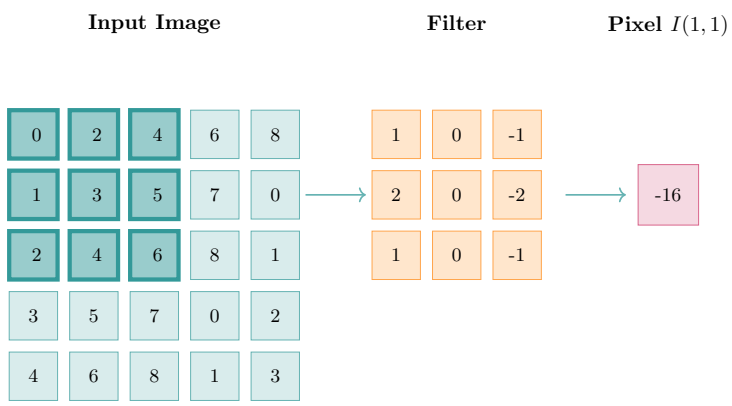

Figure 6: Example of convolution between an image and a filter.

It's important to note that the output of the convolution operation must be stored in a new image, as we need to use the original image for the next pixels.

2.8 Smoothing / Enhancement

Another type of operation that we can apply to an image is **smoothing**.

2.8.1 Low-pass filter

The idea of the low-pass filter is to remove the high-frequency components of the image, consisting of the edges and the fine details, and leaving only the low frequency components, which are typically associated with the smooth regions of the image.

For a 5×5 low-pass filter, we can define it as follows:

$$I'(c,r) = \frac{1}{25} \sum_{i=-2}^{2} \sum_{j=-2}^{2} I(c+i, r+j) \tag{2.7}$$

Each kernel value is equal to $\frac{1}{M \times N}$, where M and N are the dimensions of the kernel. This is simply the average of the pixel values in the neighborhood of the pixel we are processing.

Obviously, the bigger the kernel, the stronger the smoothing effect. However, we have to take in consideration that the pixel at the edge of the image cannot be processed. For this reason, making the kernel bigger will result in a bigger portion of the image that cannot be processed.

If wanted, we could also design a **high-pass filter** or a **band-pass filter**, by combination of filters.

2.8.2 Gaussian filter

The Gaussian filter is a low-pass filter that is defined as follows:

$$G(x, y) = ce^{-\frac{x^2+y^2}{2\sigma^2}} \tag{2.8}$$

Where c is a normalization constant to ensure integer values for the kernel, and σ is the standard deviation of the Gaussian distribution, which controls the amount of smoothing. This is an **isotropic** filter, meaning that it has the same values in all directions. This implies that, when we apply it to an image, there's no need to compute the rotation of the filter.

The effects of this filter are similar to the ones of the low-pass filter, but instead of taking the average of the pixel values in the neighborhood, it takes a weighted average, where the weights are inversely proportional to the distance from the center of the kernel. This means that the pixels closer to the center of the kernel will have a stronger influence on the output pixel value, while the pixels farther away will have a weaker influence.

2.8.3 Median filtering

The median filter is a non-linear filter that is used to remove noise from an image, especially for spike noise. If we have noise in the image, applying a low-pass or a Gaussian filter will not be effective, as the mean operator is very sensitive to outliers. On the other hand, the median operator is known to be more robust to outliers, thus it is more effective in removing noise from the image.

Instead of taking the average of the pixel values in the neighborhood, the median filter takes the median value of the pixel values in the neighborhood. The elements of the neighborhood are sorted in ascending order, and the value in the middle is taken as the output pixel value.

3 Morphology Filters

Morphology operators are a set of non-linear operations used to extract information about the shape of objects in an image. They are typically used for different tasks, such as:

- **Shape fitting**: Given a shape, check whether it fits inside an object in the image;
- **Shape extraction**: Given a shape, extract all the objects in the image that have that shape;

- **Shape analysis**: Check whether a picture has a certain shape;

Those operators are typically applied to **binary images**, where the pixel values are either 0 or 1, given a **Binary Structuring Element** (BSE), which represents the shape that we want to fit, extract or analyze.

Common operators are:

- **Erosion**: reduce the area of a certain shape in the image;
- **Dilation**: increase the area of a certain shape in the image;
- **Opening**: remove small portions of the image, close to the boundary of relevant areas;
- **Closing**: fill small holes in the image and makes region boundaries smoother.

Opening and closing are combinations of erosion and dilation.

Dilation

The structuring element S is swept over the binary image B. As the origin of the structuring element touches a pixel in the image with a value of 1, all pixels in the image that are covered by the structuring element are ORed with the value of B. This basically corresponds to sliding the structuring element over the border of the shape in the image, and adding pixels to it, thus increasing its area.

More formally, the dilation of the foreground of a binary image B, with a structuring element S is defined as follows:

$$B \oplus S = \bigcup_{b \in B} S_b \tag{3.1}$$

These operations are usually used to simplify recognition and tracking of objects in an image.

Erosion

Again, the structuring element S is swept over the binary image B. For each position where the structuring element is fully contained in the foreground of the image, the pixel in the output image are ORed with the value of B.

Closing and Opening

Closing and opening are combinations of dilation and erosion. They are defined as:

- **Closing**: first we apply **dilation** to the image, and then we apply **erosion** to the result;
- **Opening**: first we apply **erosion** to the image, and then we apply **dilation** to the result.